

CO5-AT1

Course Code: ITA0609

Course Title: Machine Learning

Student Name: S.Swagat

COMBINING AND VALIDATING TWO PANDAS DATAFRAMES

Two large Pandas DataFrames containing customer transaction and demographic data can be combined using the `merge()` function. The common column `Customer_ID` is used as the key.

The following steps are performed:

1. Combine both DataFrames using `merge()`.
2. Check for duplicate Customer IDs using filtering and grouping.
3. Sort the merged data and maintain consistent indexing.
4. Detect and debug missing or mismatched entries.
5. Use an outer merge to preserve all valid records.

SUBDIVISION 1: COMBINING THE DATAFRAMES

The transaction and demographic DataFrames are combined using `pd.merge()`. The `Customer_ID` column is used as the common key. An outer merge is used to preserve all records from both DataFrames.

```
import pandas as pd
```

```
# Transaction Data
```

```
transactions = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 105],  
    "Transaction": ["Purchase", "Purchase", "Return", "Purchase", "Purchase"],  
    "Amount": [5000, 3000, 1500, 7000, 2500]  
})
```

```
# Demographic Data
```

```
demographics = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 106],
```

```

    "Name": ["Arun", "Bala", "Charan", "Divya", "Esha"],
    "Age": [25, 30, 28, 35, 26],
    "City": ["Chennai", "Bangalore", "Chennai", "Mumbai", "Delhi"]
})

```

```

# Combine DataFrames

```

```

merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer"
)

```

```

print("Combined Data:")

```

```

print(merged_data)

```

Function used: merge() combines two DataFrames using a common column.

Why outer merge? It preserves all valid records from both DataFrames.

SUBDIVISION 2: VALIDATING DUPLICATE CUSTOMER IDS USING FILTERING AND GROUPING

After merging, duplicate Customer IDs are checked using duplicated(). The groupby() function is used to count the number of records for each Customer ID.

```

import pandas as pd

```

```

# Transaction Data

```

```

transactions = pd.DataFrame({
    "Customer_ID": [101, 102, 103, 104, 105],
    "Transaction": ["Purchase", "Purchase", "Return", "Purchase", "Purchase"],
    "Amount": [5000, 3000, 1500, 7000, 2500]
})

```

```

# Demographic Data

```

```
demographics = pd.DataFrame({
    "Customer_ID": [101, 102, 103, 104, 106],
    "Name": ["Arun", "Bala", "Charan", "Divya", "Esha"],
    "Age": [25, 30, 28, 35, 26],
    "City": ["Chennai", "Bangalore", "Chennai", "Mumbai", "Delhi"]
})
```

```
# Combine DataFrames
```

```
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer"
)
```

```
# Check duplicate Customer IDs
```

```
print("Duplicate Customer IDs:")
print(merged_data[merged_data["Customer_ID"].duplicated()])
```

```
# Group by Customer ID
```

```
print("\nCustomer ID Count:")
print(merged_data.groupby("Customer_ID").size())
```

Functions used:

`duplicated()` – finds duplicate Customer IDs.

`groupby()` – groups records based on Customer ID.

`size()` – counts records in each group.

Filtering – displays records satisfying a condition.

If every Customer ID has a count of 1, there is no duplication.

SUBDIVISION 3: SORTING AND MAINTAINING CONSISTENT INDEXING

The merged data is sorted according to `Customer_ID`. The index is then reset to maintain a continuous and consistent index.

```
import pandas as pd
```

```
# Transaction Data
```

```
transactions = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 105],  
    "Transaction": ["Purchase", "Purchase", "Return", "Purchase", "Purchase"],  
    "Amount": [5000, 3000, 1500, 7000, 2500]  
})
```

```
# Demographic Data
```

```
demographics = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 106],  
    "Name": ["Arun", "Bala", "Charan", "Divya", "Esha"],  
    "Age": [25, 30, 28, 35, 26],  
    "City": ["Chennai", "Bangalore", "Chennai", "Mumbai", "Delhi"]  
})
```

```
# Combine DataFrames
```

```
merged_data = pd.merge(  
    transactions,  
    demographics,  
    on="Customer_ID",  
    how="outer"  
)
```

```
# Sort by Customer ID
```

```
merged_data = merged_data.sort_values("Customer_ID")
```

```
# Reset index
```

```
merged_data = merged_data.reset_index(drop=True)
```

```
print("Sorted Data:")
```

```
print(merged_data)
```

```
print("\nFinal Index:")
```

```
print(merged_data.index)
```

Functions used:

sort_values() – sorts the DataFrame by Customer ID.

reset_index(drop=True) – creates a new continuous index starting from 0.

This ensures that the final DataFrame has proper ordering and consistent indexing.

SUBDIVISION 4: DEBUGGING MISMATCHED OR MISSING ENTRIES

An outer merge may produce missing values when a Customer ID exists in one DataFrame but not in the other. These missing values can be detected using isnull() and filtering.

import pandas as pd

Transaction Data

```
transactions = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 105],  
    "Transaction": ["Purchase", "Purchase", "Return", "Purchase", "Purchase"],  
    "Amount": [5000, 3000, 1500, 7000, 2500]  
})
```

Demographic Data

```
demographics = pd.DataFrame({  
    "Customer_ID": [101, 102, 103, 104, 106],  
    "Name": ["Arun", "Bala", "Charan", "Divya", "Esha"],  
    "Age": [25, 30, 28, 35, 26],  
    "City": ["Chennai", "Bangalore", "Chennai", "Mumbai", "Delhi"]  
})
```

```
# Combine DataFrames
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer"
)
```

```
# Check missing values
print("Missing Values:")
print(merged_data.isnull().sum())
```

```
# Display mismatched or missing records
print("\nMismatched or Missing Records:")
print(merged_data[merged_data.isnull().any(axis=1)])
```

Functions used:

isnull() – detects missing values.

sum() – counts missing values.

any(axis=1) – finds rows containing at least one missing value.

Filtering – displays the problematic records.

In this dataset, Customer_ID 105 is present only in the transaction data, while Customer_ID 106 is present only in the demographic data. Therefore, the outer merge helps identify these mismatched records.

CONCLUSION

Thus, Pandas merge() is used to combine the customer transaction and demographic DataFrames while preserving all valid records. duplicated() and groupby() are used to validate Customer IDs and detect duplication. sort_values() and reset_index() maintain proper ordering and consistent indexing. Finally, isnull() and filtering are used to identify and debug missing or mismatched entries.

These techniques ensure that the combined DataFrame is complete, correctly indexed, and free from unwanted duplicate Customer IDs.

2).COMPLETE DATA ANALYSIS PIPELINE USING PANDAS

A complete data analysis pipeline can be designed using Pandas to read multiple data files, process the data, perform filtering, grouping and sorting, and generate meaningful visualizations.

The pipeline consists of four main stages:

1. Loading the data
2. Processing and filtering the data
3. Grouping, aggregation and sorting
4. Generating visualizations

Debugging steps are also used to handle corrupted files and inconsistent data formats.

SUBDIVISION 1: LOADING MULTIPLE DATA FILES

Pandas `read_csv()` is used to load multiple CSV files into DataFrames. The `concat()` function combines the files into a single DataFrame.

```
import pandas as pd
```

```
# File 1
```

```
data1 = pd.DataFrame({  
    "Product": ["Laptop", "Phone", "Tablet"],  
    "Category": ["Electronics", "Electronics", "Electronics"],  
    "Sales": [55000, 30000, 20000],  
    "Quantity": [5, 8, 6]  
})
```

```
# File 2
```

```
data2 = pd.DataFrame({  
    "Product": ["Monitor", "Keyboard", "Mouse"],  
    "Category": ["Electronics", "Accessories", "Accessories"],  
    "Sales": [18000, 5000, 3000],  
    "Quantity": [6, 10, 15]  
})
```

```
# Combine multiple files  
data = pd.concat([data1, data2], ignore_index=True)
```

```
print("Combined Data:")  
print(data)
```

Functions Used

- `read_csv()` → reads CSV files into Pandas DataFrames.
- `concat()` → combines multiple DataFrames.
- `ignore_index=True` → creates a continuous index.

Optimization

For large files, only required columns can be loaded using the `usecols` parameter. This reduces memory usage and improves file-reading speed.

SUBDIVISION 2: PROCESSING AND FILTERING DATA

After loading the data, the required records can be selected using filtering conditions. User-defined conditions can be applied to select specific records.

```
import pandas as pd
```

```
# File 1
```

```
data1 = pd.DataFrame({  
    "Product": ["Laptop", "Phone", "Tablet"],  
    "Category": ["Electronics", "Electronics", "Electronics"],  
    "Sales": [55000, 30000, 20000],  
    "Quantity": [5, 8, 6]  
})
```

```
# File 2
```

```
data2 = pd.DataFrame({  
    "Product": ["Monitor", "Keyboard", "Mouse"],  
    "Category": ["Electronics", "Accessories", "Accessories"],
```



```

    "Sales": [18000, 5000, 3000],
    "Quantity": [6, 10, 15]
})

# Combine data
data = pd.concat([data1, data2], ignore_index=True)

# User-defined condition
filtered_data = data[data["Sales"] > 10000]

print("Filtered Data:")
print(filtered_data)

```

Here, the condition `Sales > 10000` selects only products whose sales are greater than 10,000.

Functions Used

- Boolean filtering → selects records based on conditions.
- `data["Sales"] > 10000` → creates a filtering condition.
- `data[condition]` → displays matching records.

SUBDIVISION 3: GROUPING, AGGREGATION AND SORTING

The data can be grouped according to category and aggregated using functions such as `sum()`.

import pandas as pd

```

# File 1

data1 = pd.DataFrame({
    "Product": ["Laptop", "Phone", "Tablet"],
    "Category": ["Electronics", "Electronics", "Electronics"],
    "Sales": [55000, 30000, 20000],
    "Quantity": [5, 8, 6]
})

```

```

# File 2

data2 = pd.DataFrame({
    "Product": ["Monitor", "Keyboard", "Mouse"],
    "Category": ["Electronics", "Accessories", "Accessories"],
    "Sales": [18000, 5000, 3000],
    "Quantity": [6, 10, 15]
})

# Combine data

data = pd.concat([data1, data2], ignore_index=True)

# Group by Category

grouped_data = data.groupby("Category")["Sales"].sum()

# Sort the result

grouped_data = grouped_data.sort_values(ascending=False)

print("Category-wise Sales:")
print(grouped_data)

```

Functions Used

- `groupby()` → groups data according to Category.
- `sum()` → calculates total sales.
- `sort_values()` → sorts the aggregated result.
- `ascending=False` → sorts from highest to lowest.

Grouping and sorting help identify which category has the highest sales.

SUBDIVISION 4: GENERATING MEANINGFUL VISUALIZATIONS

Matplotlib can be used to visualize the aggregated data. A bar chart is suitable for comparing sales between categories.

```

import pandas as pd

import matplotlib.pyplot as plt

```

```
# File 1
```

```
data1 = pd.DataFrame({  
    "Product": ["Laptop", "Phone", "Tablet"],  
    "Category": ["Electronics", "Electronics", "Electronics"],  
    "Sales": [55000, 30000, 20000],  
    "Quantity": [5, 8, 6]  
})
```

```
# File 2
```

```
data2 = pd.DataFrame({  
    "Product": ["Monitor", "Keyboard", "Mouse"],  
    "Category": ["Electronics", "Accessories", "Accessories"],  
    "Sales": [18000, 5000, 3000],  
    "Quantity": [6, 10, 15]  
})
```

```
# Combine data
```

```
data = pd.concat([data1, data2], ignore_index=True)
```

```
# Group sales by category
```

```
grouped_data = data.groupby("Category")["Sales"].sum()
```

```
# Plot
```

```
grouped_data.plot(kind="bar")
```

```
plt.title("Category-wise Sales")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Total Sales")
```

```
plt.show()
```

Functions Used

- `plot(kind="bar")` → creates a bar chart.
- `title()` → adds a title.
- `xlabel()` → labels the X-axis.
- `ylabel()` → labels the Y-axis.
- `show()` → displays the graph.

The bar chart provides a meaningful comparison of total sales between categories.

DEBUGGING CORRUPTED FILES AND INCONSISTENT DATA

When working with multiple files, some files may be corrupted or have inconsistent formats. The following steps can be used for debugging:

1. Check whether the file exists before reading it.
2. Verify that all files have the required columns.
3. Check for missing values using `isnull()`.
4. Check data types using `dtypes`.
5. Handle corrupted files using `try-except`.
6. Remove or correct invalid records.
7. Ensure that column names are consistent before combining the files.

Example:

```
import pandas as pd
```

```
try:
```

```
    data = pd.read_csv("sales.csv")  
    print("File loaded successfully")
```

```
    print(data.dtypes)  
    print(data.isnull().sum())
```

```
except Exception as e:
```

```
    print("Error while reading file:", e)
```

The try-except block prevents the program from stopping when an error occurs and helps identify the problem.

OPTIMIZATION TECHNIQUES

The pipeline can be optimized using the following methods:

- Load only required columns using usecols.
- Use appropriate data types to reduce memory usage.
- Use concat() instead of manually combining rows.
- Perform filtering before grouping when possible.
- Sort only when sorting is required.
- Avoid unnecessary copies of DataFrames.
- Handle missing and invalid data before analysis.

CONCLUSION

Thus, a complete Pandas data analysis pipeline starts by loading multiple files into DataFrames, combining and cleaning the data, applying user-defined filtering conditions, grouping and aggregating the data, sorting the results, and finally generating meaningful visualizations.

Debugging techniques such as checking missing values, verifying data types, checking column consistency, and using try-except help handle corrupted files and inconsistent data formats. Efficient file loading, filtering, grouping, and sorting improve the performance and reliability of the complete data analysis pipeline.